Design and Development of a Rule-Based AI-Powered Student Productivity and Academic Scheduling System

APT 3065A – Project Coding Stage Report

Student Name: Victor Chege

Student ID: 662833

Instructor: Prof. Fredrick Michael Ogore

Spring Semester 2026

Table of Contents

Project Overview 2

Brief recap of the project goal and scope 3

Key features planned for the final product 3

Development Progress 3

Modules / components already implemented 4

Backend Environment Setup 4

Database Model Implementation 5

Django Admin Integration 5

Initial AI Prioritization Logic 5

Screenshots of progress 6

Technical Details 8

Programming languages, frameworks, and tools used 9

System architecture diagram 9

Database schema / API design 10

Testing and Validation 11

Challenges and How They Will Be Overcome 12

Collaboration Environment 13

Next Steps 13

References 14

Project Overview

Brief recap of the project goal and scope

The objective of this project is to design and develop a web-based student productivity system that assists university students in organizing academic tasks and generating structured study schedules. The system integrates rule-based Artificial Intelligence logic to prioritize academic tasks based on measurable attributes such as deadline proximity and task difficulty.

University students frequently face challenges managing overlapping assignments, projects, quizzes, and examinations. While digital productivity tools such as task managers and calendar systems exist, these tools require students to manually determine task priority and study plans. The proposed system addresses this challenge by implementing an automated prioritization and scheduling mechanism that functions as an academic decision-support tool.

The system is designed as a full-stack web application consisting of a frontend user interface, a backend server responsible for system logic, and a database used to store user and task information. The backend implements rule-based logic that evaluates task urgency and complexity to generate optimized study schedules.

The system therefore aims to improve academic time management and productivity by assisting students in planning their workload more effectively.

Key features planned for the final product

The final system will include the following core features:

- Secure user registration and login
- Academic task creation and management
- Task prioritization using rule-based AI logic
- Study availability configuration by the student

- Automatic generation of optimized study schedules
- Dynamic schedule updates when tasks change
- Dashboard for viewing tasks and generated schedules

These features collectively enable the system to act as an intelligent academic planning assistant for university students.

Development Progress

The coding stage of the project began with the implementation of the backend environment and the core system architecture defined during the system design phase. The main objective of this stage was to establish a stable development environment, implement the database structure, and begin implementing the AI prioritization logic.

Modules / components already implemented

Backend Environment Setup

The backend system was implemented using the Django web framework. A Python virtual environment was created to isolate project dependencies and ensure compatibility between required libraries.

The backend project structure was organized into modular components to improve maintainability and scalability.

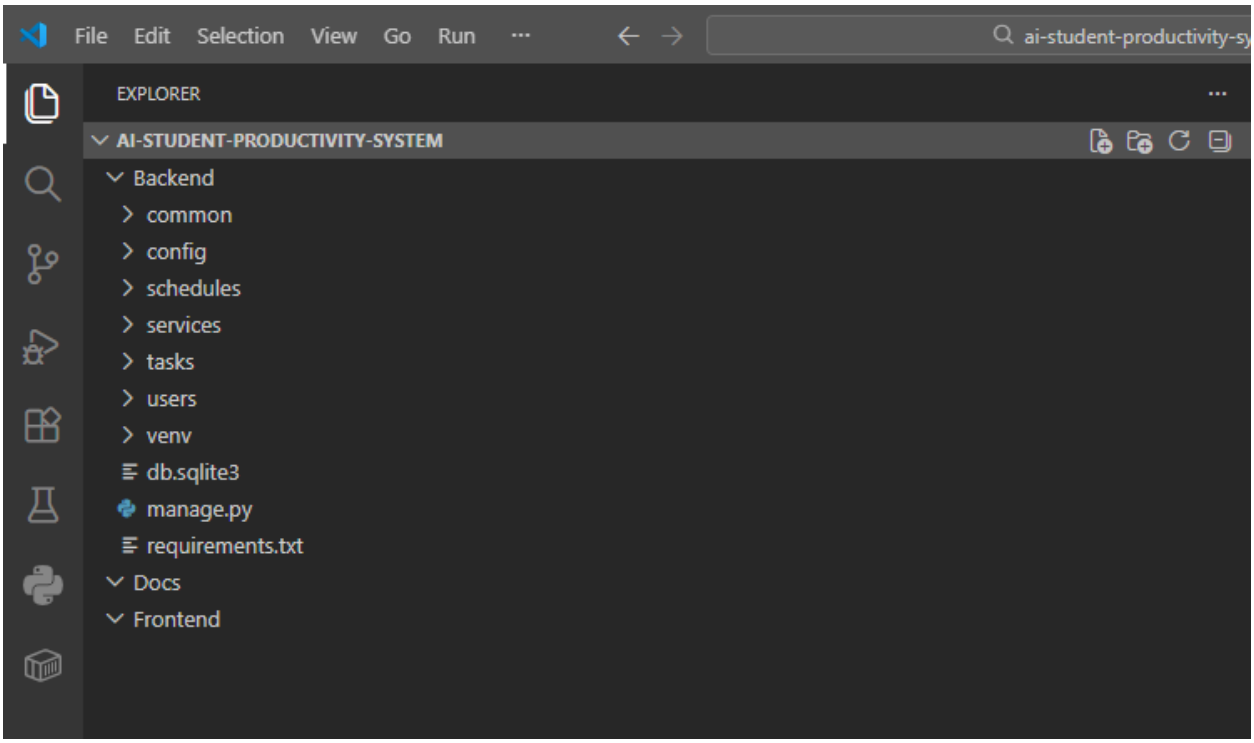

Figure 1: Django Backend Project Structure in Visual Studio Code Explorer

The modular architecture separates system responsibilities into independent applications.

- **Users module:** manages user authentication and account information
- **Tasks module:** handles task creation, updating, deletion, and completion
- **Schedules module:** manages schedule generation and storage
- **Services module:** contains the rule-based prioritization logic

Database Model Implementation

The core database models were implemented according to the system design defined earlier in the project.

The following core entities were implemented based on the system design:

- User

- Task
- StudyAvailability
- Schedule
- ScheduleItem

These models define the relational structure of the system database and represent the primary components required for task prioritization and schedule generation.

The models were migrated successfully to the database using Django migration tools.

Django Admin Integration

All models were registered in the Django administrative interface. This allows administrators to interact with the database and test system functionality.

Through the admin panel it is possible to:

- Create and manage tasks
- Define study availability periods
- Create schedules
- View generated schedule items

This feature allowed early testing of database functionality before implementing the frontend interface.

Initial AI Prioritization Logic

The first version of the rule-based prioritization engine was implemented in the backend service layer.

The prioritization logic calculates a **priority score** for each task based on:

- Deadline proximity (urgency)
- Task difficulty (complexity)

The prioritization formula implemented in the system is:

Priority Score = (Urgency Weight × Deadline Proximity) + (Difficulty Weight × Task Complexity)

This algorithm ensures that tasks with closer deadlines and higher difficulty levels receive higher priority scores.

The prioritization logic was successfully tested using the Django interactive shell.

Screenshots of progress

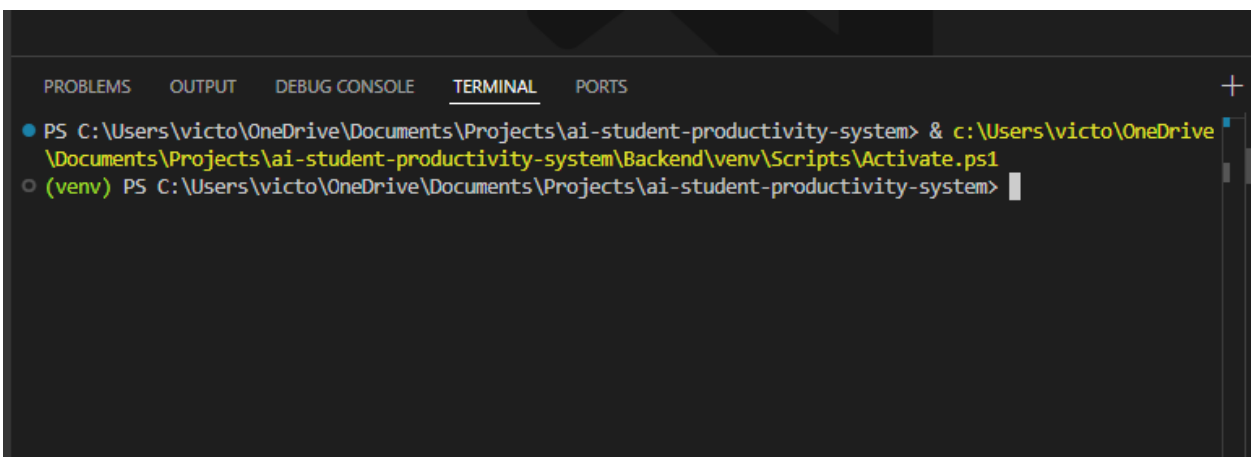

Figure 2: Activated Python Virtual Environment

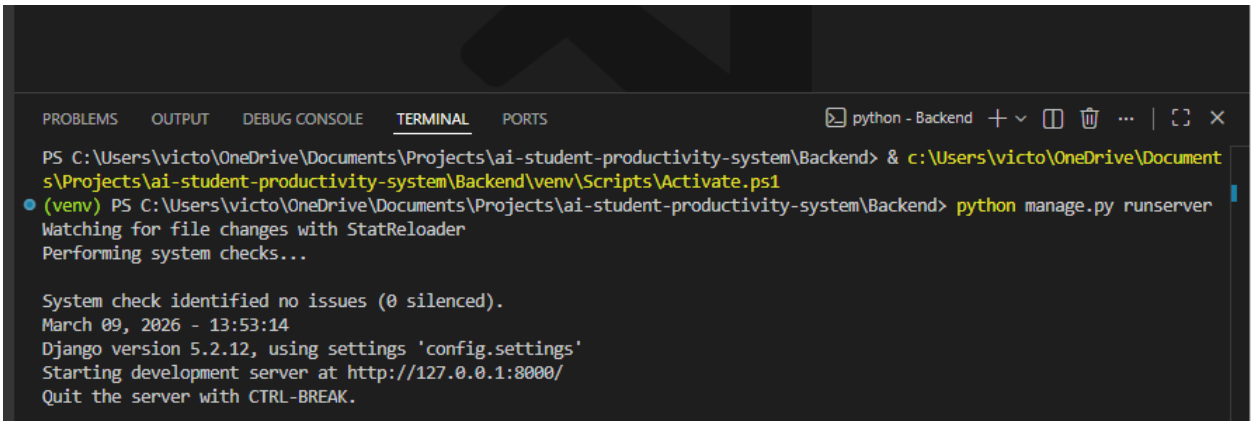

Figure 3: Django Development Server Running Locally.

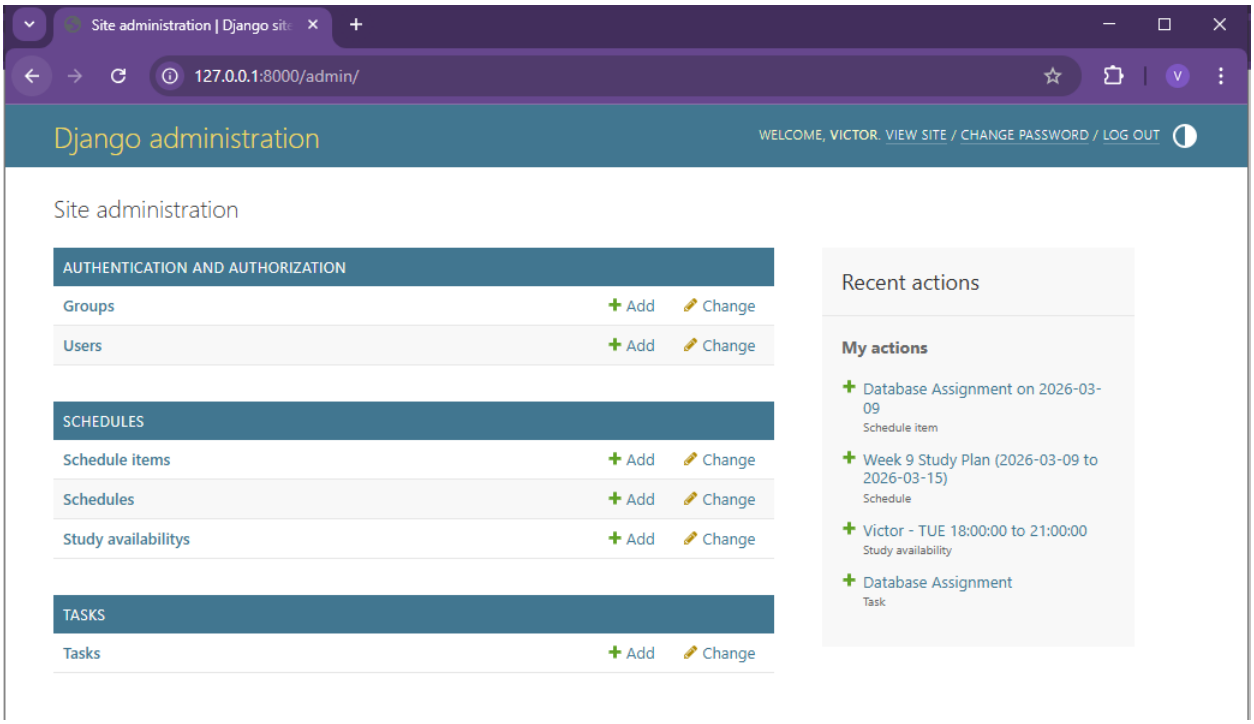

Figure 4: Django administrative interface showing system models.

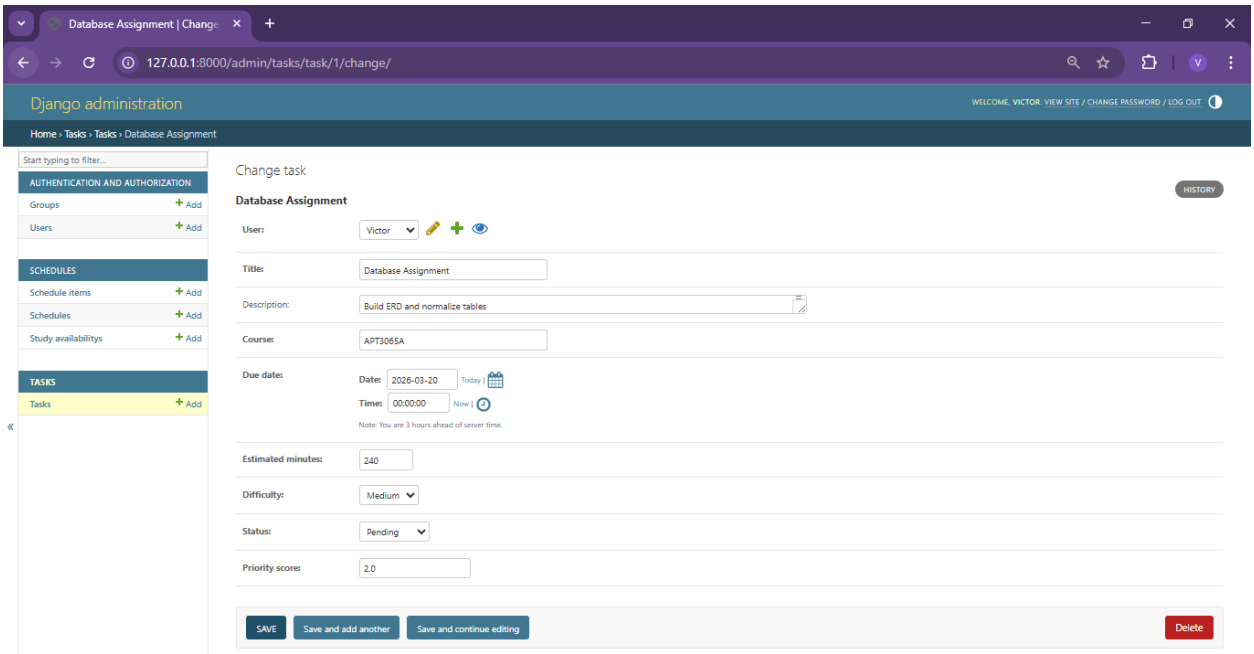

Figure 5: Example task stored in the system database.

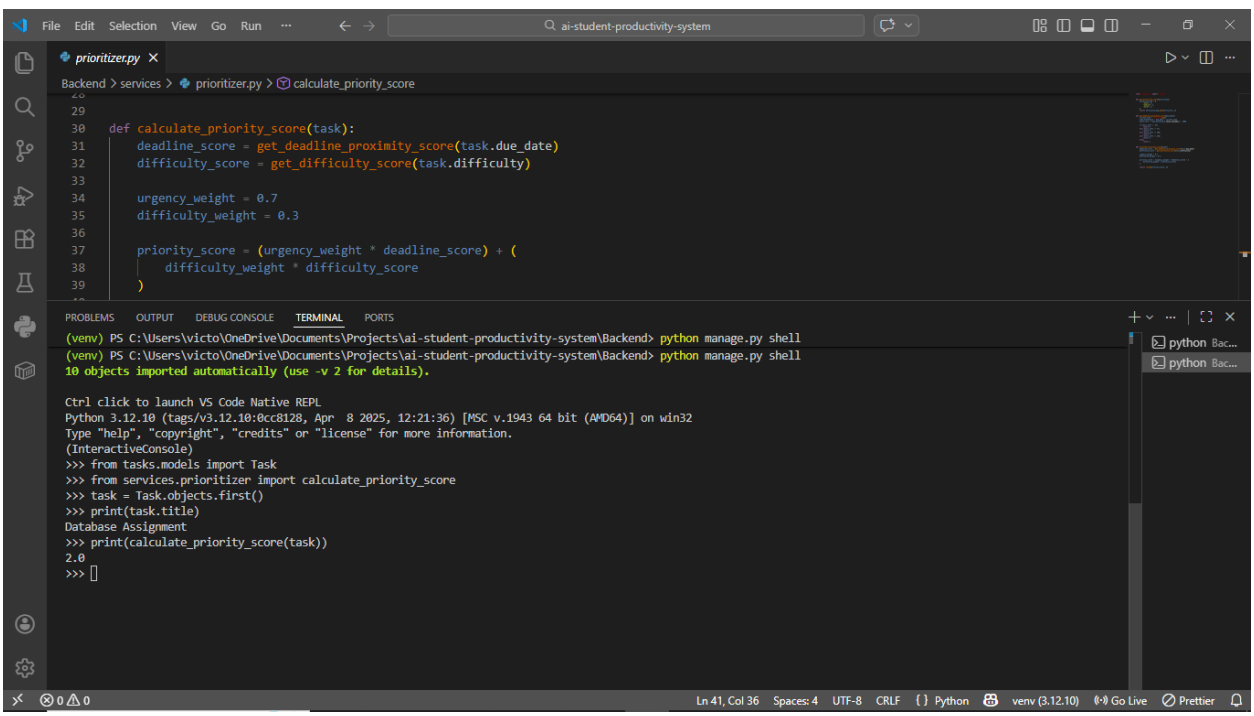

Figure 6: Task priority score calculated using rule-based prioritization logic.

Technical Details

Programming languages, frameworks, and tools used

The system was developed using a modern full-stack architecture that integrates several technologies to support backend processing, database management, and future frontend interaction. The primary programming languages used in the development process are Python and JavaScript. Python was selected because it integrates effectively with the Django framework and provides strong support for implementing rule-based logic within the backend system. JavaScript will be used to develop the interactive frontend interface that communicates with the backend services.

The backend of the system was implemented using the Django web framework together with Django REST Framework. Django provides a robust environment for developing secure and scalable web applications, while Django REST Framework enables the creation of RESTful APIs that allow the frontend interface to communicate with the backend services. The frontend interface of the system will be implemented using the React framework. Although the frontend development has not yet begun at this stage of the project, React was selected because it supports the development of responsive and dynamic user interfaces.

Regarding data storage, the system is designed to use PostgreSQL as the primary database management system due to its reliability, scalability, and strong support for relational data structures. However, during the early stages of development, SQLite was used temporarily because it is automatically included with Django and allows rapid testing and prototyping without requiring additional configuration.

Several development tools will also be used during the coding stage of the project. Visual Studio Code served as the primary development environment because it provides code editing, debugging tools, and useful extensions for Python and Django development. Git and GitHub will be used for version control and code management, allowing the project to maintain a structured development history and secure backup of source code. Postman will be used to test API endpoints during backend development, while the Django administrative interface was used to test database operations such as creating and modifying tasks, schedules, and other system entities.

These technologies were selected because they provide scalability, security, and compatibility with rule-based Artificial Intelligence logic while supporting a modular and maintainable system architecture.

System architecture diagram

The system follows a three-tier architecture consisting of three main layers: the presentation layer, the application layer, and the data layer. This architecture separates system responsibilities into independent components, improving scalability, maintainability, and system performance.

The presentation layer represents the user interface of the system and will be implemented using React. This layer allows students to interact with the system by creating tasks, viewing schedules, and managing their academic workload. The application layer represents the backend server and is implemented using Django. This layer contains the core system logic, including authentication management, task processing, rule-based task prioritization, and schedule generation. The data layer is implemented using a relational database that stores system information such as users, tasks, study availability periods, schedules, and schedule items.

Communication between system components occurs through RESTful APIs. The React frontend sends requests to the Django backend through API endpoints, and the backend processes these requests, retrieves or updates information in the database, and returns responses to the frontend interface. This layered architecture ensures clear separation between system components and allows each layer to be developed and maintained independently.

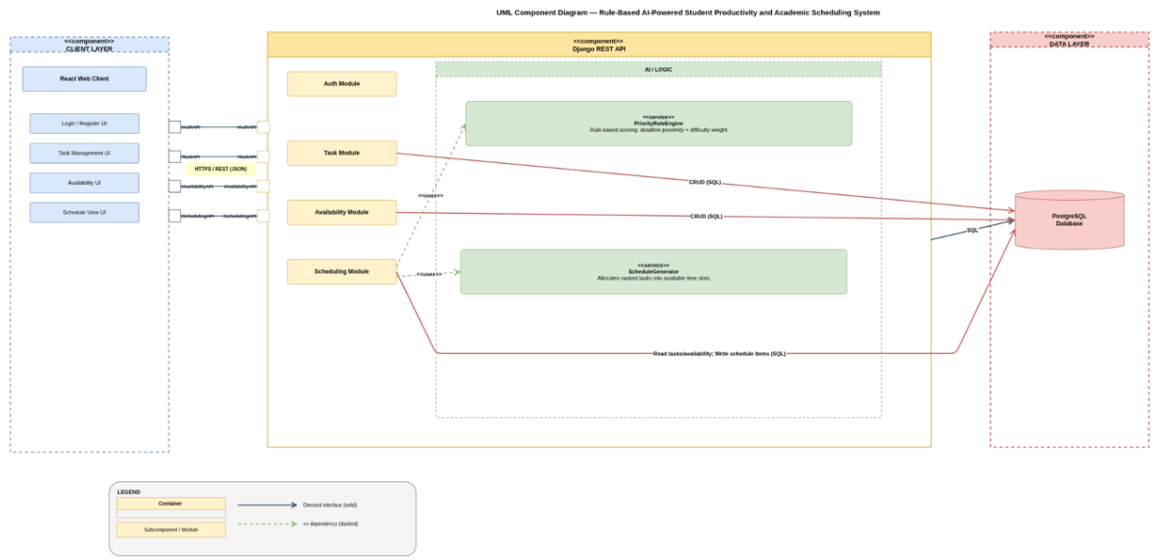

Figure 7: System Architecture Component Diagram

Database schema / API design

The database structure of the system is designed around five primary tables that represent the core entities of the student productivity system. These tables include Users, Tasks, Study Availability, Schedules, and Schedule Items. Each table stores specific information required for task management and automated scheduling.

The Users table stores account information for students using the system. The Tasks table stores academic tasks created by users, including attributes such as task title, description, deadline, difficulty level, estimated completion time, and the calculated priority score generated by the rule-based prioritization algorithm. This table represents the primary input for the scheduling system.

The Study Availability table stores the time periods during which a student is available to study. This information is used by the scheduling engine to allocate study sessions. The Schedules table stores generated study schedules created by the system, while the Schedule Items table stores the individual study sessions assigned to tasks within a specific schedule.

This relational database structure ensures data integrity and supports efficient retrieval and management of information required for automated task prioritization and study schedule generation.

Testing and Validation

Testing during this stage of development focused primarily on validating the backend functionality and ensuring that the core system components operate correctly. Since the frontend interface has not yet been implemented, testing was conducted using the Django administrative interface and the Django interactive shell. These tools allowed verification of database operations and confirmation that the implemented logic performs as expected.

The first stage of testing involved verifying that the database models were correctly created and migrated. Using Django's migration system, the database tables corresponding to the system entities were successfully generated. These tables include Users, Tasks, Study Availability, Schedules, and Schedule Items. The Django Admin interface was then used to manually create and manage records within these tables to confirm that the database structure functions correctly.

The second stage of testing focused on validating the rule-based prioritization logic implemented within the backend service layer. Tasks with different deadlines and difficulty levels were created and tested using the Django shell environment. The prioritization function successfully computed priority scores based on the weighted formula defined in the system design. These results confirmed that the system can evaluate tasks and assign priority scores accurately according to the defined rules.

Although these tests confirmed that the backend components are functioning correctly, the system is still in an early development stage. The scheduling engine responsible for automatically allocating study sessions has not yet been fully implemented. Additionally, the frontend user interface has not yet been integrated with the backend APIs. These components will be tested during later stages of the development process when full system integration is performed.

Challenges and How They Will Be Overcome

During the early coding stage of the project, several challenges were encountered while setting up the development environment and implementing the backend architecture. These challenges primarily related to environment configuration, dependency management, and backend structure design.

One of the initial challenges involved configuring the development environment and ensuring compatibility between the required software components. Installing and configuring Python, Django, and other necessary libraries required troubleshooting to ensure that all packages were properly installed and functioning correctly within the project environment. This issue was resolved by creating a Python virtual environment, which isolates project dependencies and prevents conflicts between different Python packages.

Another challenge involved designing the backend architecture in a way that aligns with the system design defined earlier in the project. Because the system consists of multiple functional components, careful planning was required to divide the backend into modular applications such as users, tasks, and schedules. This modular architecture was implemented using Django apps, allowing different parts of the system to be developed independently while maintaining clear separation of responsibilities.

Dependency management also presented a challenge during development. Ensuring that required libraries were installed within the virtual environment and that the correct versions were used required careful configuration. This issue was addressed by installing all required packages within the virtual environment and documenting dependencies in a requirements file, ensuring that the project environment can be reproduced easily in the future.

These challenges were resolved through careful configuration, adherence to recommended development practices, and incremental testing of system components. As development continues, the use of modular design and version control will help reduce complexity and prevent similar issues from affecting future stages of the project.

Collaboration Environment

The development of the system was carried out using a local development environment on a personal computer. Visual Studio Code was used as the primary integrated development environment (IDE) for writing, editing, and organizing the source code. The editor provided useful features such as syntax highlighting, debugging tools, and extensions that support Python and Django development.

The backend server was developed using the Django framework and tested locally using the Django development server. Database operations were tested using the Django administrative interface, which allowed manual creation and modification of tasks, schedules, and other system entities during the development process.

The Django interactive shell was also used to test backend logic, particularly the rule-based task prioritization algorithm. Through the shell environment, tasks stored in the database were retrieved and processed by the prioritization function to confirm that priority scores were calculated correctly.

Development is currently conducted in a local environment using Visual Studio Code. Version control tools such as Git and GitHub will be integrated in later stages of development to support structured code management and system deployment.

Next Steps

The next stage of development will focus on implementing the remaining core functionalities required to complete the system. While the backend environment and database models have been successfully implemented, several critical components still need to be developed before the system can operate as a fully functional application.

The first major task in the next phase of development will involve implementing the scheduling engine. This component will retrieve prioritized tasks from the database and allocate them into

available study time blocks defined by the user. The scheduling algorithm will ensure that tasks with higher priority scores are scheduled earlier while preventing conflicts between study sessions.

The second task will involve completing the REST API endpoints required for communication between the frontend and backend components. These endpoints will allow the frontend application to perform operations such as creating tasks, updating tasks, retrieving schedules, and generating study plans.

Following the completion of the backend functionality, frontend development will begin using the React framework. The frontend will provide user interfaces for registration, task management, study availability input, and schedule visualization. Once the frontend is implemented, full system integration will be performed to ensure that the frontend interface communicates correctly with the backend APIs.

The final stage of the project will involve system testing, debugging, and performance evaluation. This stage will ensure that all system components function correctly together and that the application provides accurate and reliable scheduling recommendations to users.

Based on the current development progress, approximately 40 percent of the system implementation has been completed. The remaining development effort will focus primarily on implementing the scheduling algorithm, building the frontend interface, and performing system integration and testing before the final project submission.

References

Araka, E., Maina, E., Gitonga, R., Oboko, R., & Kihoro, J. (2021). University students' perception of the usefulness of learning management system features in promoting self-regulated learning in online environments. *International Journal of Education and Development Using Information and Communication Technology*, 17(1), 45–64.

Chege, V. (2026). *Design and development of a rule-based AI-powered student productivity and academic scheduling system: Project proposal*. Unpublished project proposal submitted to APT 3065A, United States International University–Africa.

Chege, V. (2026). *System design documentation for a rule-based AI-powered student productivity and academic scheduling system*. Unpublished system design document submitted to APT 3065A, United States International University–Africa.

Django Software Foundation. (2024). *Django documentation*. https://docs.djangoproject.com

Django REST Framework. (2024). *Django REST framework documentation*. https://www.django-rest-framework.org/

Holmes, W., Bialik, M., & Fadel, C. (2023). *Artificial intelligence in education: Recent developments and future implications*. Center for Curriculum Redesign.

React Team. (2024). *React documentation*. https://react.dev